

```

82. int table[10];
    for (int x = 0; x < 20; x++)
    {
        cout << "Enter the next value: ";
        cin >> table[x];
    }

83. int hours[3] = 8, 12, 16;
84. int numbers[8] = {1, 2, , 4, , 5};
85. float ratings[];
86. char greeting[] = {'H', 'e', 'l', 'l', 'o'};
    cout << greeting;
87. int array1[4], array2[4] = {3, 6, 9, 12};
    array1 = array2;
88. void showValues(int nums)
    {
        for (int count = 0; count < 8; count++)
            cout << nums[count];
    }
89. void showValues(int nums[4][])
    {
        for (rows = 0; rows < 4; rows++)
            for (cols = 0; cols < 5; cols++)
                cout << nums[rows][cols];
    }
90. vector<int> numbers = { 1, 2, 3, 4 };

```

Programming Challenges

1. Largest/Smallest Array Values

Write a program that lets the user enter 10 values into an array. The program should then display the largest and smallest values stored in the array.

2. Rainfall Statistics

Write a program that lets the user enter the total rainfall for each of 12 months into an array of doubles. The program should calculate and display the total rainfall for the year, the average monthly rainfall, and the months with the highest and lowest amounts.

Input Validation: Do not accept negative numbers for monthly rainfall figures.

3. Chips and Salsa

Write a program that lets a maker of chips and salsa keep track of sales for five different types of salsa: mild, medium, sweet, hot, and zesty. The program should use two parallel 5-element arrays: an array of strings that holds the five salsa names and an array of integers that holds the number of jars sold during the past month for each salsa type. The salsa names should be stored using an initialization list at the time the name array is created. The program should prompt the user to enter the number of jars

sold for each type. Once this sales data has been entered, the program should produce a report that displays sales for each salsa type, total sales, and the names of the highest selling and lowest selling products.

Input Validation: Do not accept negative values for number of jars sold.

4. Larger Than n

In a program, write a function that accepts three arguments: an array, the size of the array, and a number n . Assume that the array contains integers. The function should display all of the numbers in the array that are greater than the number n .

5. Monkey Business

A local zoo wants to keep track of how many pounds of food each of its three monkeys eats each day during a typical week. Write a program that stores this information in a two-dimensional 3×5 array, where each row represents a different monkey and each column represents a different day of the week. The program should first have the user input the data for each monkey. Then it should create a report that includes the following information:

- Average amount of food eaten per day by the whole family of monkeys.
- The least amount of food eaten during the week by any one monkey.
- The greatest amount of food eaten during the week by any one monkey.

Input Validation: Do not accept negative numbers for pounds of food eaten.

6. Rain or Shine

An amateur meteorologist wants to keep track of weather conditions during the past year's three-month summer season and has designated each day as either rainy ('R'), cloudy ('C'), or sunny ('S'). Write a program that stores this information in a 3×30 array of characters, where the row indicates the month (0 = June, 1 = July, 2 = August) and the column indicates the day of the month. Note that data are not being collected for the 31st of any month. The program should begin by reading the weather data in from a file. Then it should create a report that displays, for each month and for the whole three-month period, how many days were rainy, how many were cloudy, and how many were sunny. It should also report which of the three months had the largest number of rainy days. Data for the program can be found in the `RainOrShine.txt` file.

7. Number Analysis Program

Write a program that asks the user for a file name. Assume the file contains a series of numbers, each written on a separate line. The program should read the contents of the file into an array and then display the following data:

- The lowest number in the array
- The highest number in the array
- The total of the numbers in the array
- The average of the numbers in the array

If you have downloaded this book's source code from the companion Web site, you will find a file named `numbers.txt` in the Chapter 07 folder. You can use the file to test the program. (The companion Web site is at www.pearsonhighered.com/gaddis.)

8. Lo Shu Magic Square

The Lo Shu Magic Square is a grid with 3 rows and 3 columns shown in Figure 7-19. The Lo Shu Magic Square has the following properties:

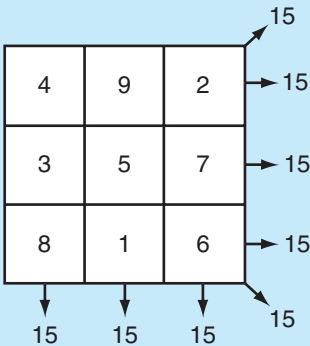
- The grid contains the numbers 1 through 9 exactly.
- The sum of each row, each column, and each diagonal all add up to the same number. This is shown in Figure 7-20.

In a program you can simulate a magic square using a two-dimensional array. Write a function that accepts a two-dimensional array as an argument, and determines whether the array is a Lo Shu Magic Square. Test the function in a program.

Figure 7-19

4	9	2
3	5	7
8	1	6

Figure 7-20



9. Payroll

Write a program that uses the following arrays:

- empId: an array of seven long integers to hold employee identification numbers. The array should be initialized with the following numbers:

```
5658845  4520125  7895122  8777541
8451277  1302850  7580489
```

- hours: an array of seven integers to hold the number of hours worked by each employee
- payRate: an array of seven doubles to hold each employee’s hourly pay rate
- wages: an array of seven doubles to hold each employee’s gross wages

The program should relate the data in each array through the subscripts. For example, the number in element 0 of the hours array should be the number of hours worked by

the employee whose identification number is stored in element 0 of the `empId` array. That same employee's pay rate should be stored in element 0 of the `payRate` array.

The program should display each employee number and ask the user to enter that employee's hours and pay rate. It should then calculate the gross wages for that employee (hours times pay rate) and store them in the `wages` array. After the data has been entered for all the employees, the program should display each employee's identification number and gross wages.

Input Validation: Do not accept negative values for hours or numbers less than 15.00 for pay rate.

10. Driver's License Exam

The local Driver's License Office has asked you to write a program that grades the written portion of the driver's license exam. The exam has 20 multiple choice questions. Here are the correct answers:

- | | | | |
|------|-------|-------|-------|
| 1. A | 6. B | 11. A | 16. C |
| 2. D | 7. A | 12. C | 17. C |
| 3. B | 8. B | 13. D | 18. A |
| 4. B | 9. C | 14. B | 19. D |
| 5. C | 10. D | 15. D | 20. B |

Your program should store the correct answers shown above in an array. It should ask the user to enter the student's answers for each of the 20 questions, and the answers should be stored in another array. After the student's answers have been entered, the program should display a message indicating whether the student passed or failed the exam. (A student must correctly answer 15 of the 20 questions to pass the exam.) It should then display the total number of correctly answered questions, the total number of incorrectly answered questions, and a list showing the question numbers of the incorrectly answered questions.

Input Validation: Only accept the letters A, B, C, or D as answers.

11. Exam Grader

One of your professors has asked you to write a program to grade her final exams, which consist of only 20 multiple-choice questions. Each question has one of four possible answers: A, B, C, or D. The file `CorrectAnswers.txt` contains the correct answers for all of the questions, with each answer written on a separate line. The first line contains the answer to the first question, the second line contains the answer to the second question, and so forth. (Download the book's source code from the companion Web site at www.pearsonhighered.com/gaddis. You will find the file in the Chapter 07 folder.)

Write a program that reads the contents of the `CorrectAnswers.txt` file into a `char` array, and then reads the contents of another file, containing a student's answers, into a second `char` array. (You can use the file `StudentAnswers.txt` for testing purposes. This file is also in the Chapter 07 source code folder, available on the book's companion Web site.) The program should determine the number of questions that the student missed and then display the following:

- A list of the questions missed by the student, showing the correct answer and the incorrect answer provided by the student for each missed question

- The total number of questions missed
- The percentage of questions answered correctly. This can be calculated as

$$\text{Correctly Answered Questions} \div \text{Total Number of Questions}$$

- If the percentage of correctly answered questions is 70% or greater, the program should indicate that the student passed the exam. Otherwise, it should indicate that the student failed the exam.

12. Grade Book

A teacher has five students who have taken four tests. The teacher uses the following grading scale to assign a letter grade to a student, based on the average of his or her four test scores.

Test Score	Letter Grade
90–100	A
80–89	B
70–79	C
60–69	D
0–59	F

Write a program that uses an array of `string` objects to hold the five student names, an array of five characters to hold the five students' letter grades, and five arrays of four `doubles` to hold each student's set of test scores.

The program should allow the user to enter each student's name and his or her four test scores. It should then calculate and display each student's average test score and a letter grade based on the average.

Input Validation: Do not accept test scores less than 0 or greater than 100.

13. Grade Book Modification

Modify the grade book application in Programming Challenge 13 so it drops each student's lowest score when determining the test score averages and letter grades.

14. Lottery Application

Write a program that simulates a lottery. The program should have an array of five integers named `lottery` and should generate a random number in the range of 0 through 9 for each element in the array. The user should enter five digits, which should be stored in an integer array named `user`. The program is to compare the corresponding elements in the two arrays and keep a count of the digits that match. For example, the following shows the `lottery` array and the `user` array with sample numbers stored in each. There are two matching digits (elements 2 and 4).

lottery array:

7	4	9	1	3
---	---	---	---	---

user array:

4	2	9	7	3
---	---	---	---	---

The program should display the random numbers stored in the `lottery` array and the number of matching digits. If all of the digits match, display a message proclaiming the user as a grand prize winner.

15. **vector** Modification

Modify the National Commerce Bank case study presented in Program 7-23 so `pin1`, `pin2`, and `pin3` are **vectors** instead of arrays. You must also modify the `testPIN` function to accept a **vector** instead of an array.

16. World Series Champions

If you have downloaded this book's source code from the companion Web site, you will find the following files in this chapter's folder:

- `Teams.txt`—This file contains a list of several Major League baseball teams in alphabetical order. Each team listed in the file has won the World Series at least once.
- `WorldSeriesWinners.txt`—This file contains a chronological list of the World Series' winning teams from 1903 to 2012. (The first line in the file is the name of the team that won in 1903, and the last line is the name of the team that won in 2012. Note that the World Series was not played in 1904 or 1994.)

Write a program that displays the contents of the `Teams.txt` file on the screen and prompts the user to enter the name of one of the teams. The program should then display the number of times that team has won the World Series in the time period from 1903 to 2012.



TIP: Read the contents of the `WorldSeriesWinners.txt` file into an array or **vector**. When the user enters the name of a team, the program should step through the array or **vector** counting the number of times the selected team appears.

17. Name Search

If you have downloaded this book's source code from the companion Web site, you will find the following files in this chapter's folder:

- `GirlNames.txt`—This file contains a list of the 200 most popular names given to girls born in the United States from 2000 to 2009.
- `BoyNames.txt`—This file contains a list of the 200 most popular names given to boys born in the United States from 2000 to 2009.

Write a program that reads the contents of the two files into two separate arrays or **vectors**. The user should be able to enter a boy's name, a girl's name, or both, and the application should display messages indicating whether the names were among the most popular.

18. Tic-Tac-Toe Game

Write a program that allows two players to play a game of tic-tac-toe. Use a two-dimensional `char` array with three rows and three columns as the game board. Each element of the array should be initialized with an asterisk (*). The program should run a loop that

- Displays the contents of the board array
- Allows player 1 to select a location on the board for an X. The program should ask the user to enter the row and column number.

- Allows player 2 to select a location on the board for an O. The program should ask the user to enter the row and column number.
- Determines whether a player has won, or a tie has occurred. If a player has won, the program should declare that player the winner and end. If a tie has occurred, the program should say so and end.

Player 1 wins when there are three Xs in a row on the game board. The Xs can appear in a row, in a column, or diagonally across the board. A tie occurs when all of the locations on the board are full, but there is no winner.

19. 2D Array Operations

Write a program that creates a two-dimensional array initialized with test data. Use any data type you wish. The program should have the following functions:

- **getTotal**. This function should accept a two-dimensional array as its argument and return the total of all the values in the array.
- **getAverage**. This function should accept a two-dimensional array as its argument and return the average of all the values in the array.
- **getRowTotal**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the total of the values in the specified row.
- **getColumnTotal**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a column in the array. The function should return the total of the values in the specified column.
- **getHighestInRow**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the highest value in the specified row of the array.
- **getLowestInRow**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the lowest value in the specified row of the array.

Demonstrate each of the functions in this program.

Group Project

20. Theater Seating

This program should be designed and written by a team of students. Here are some suggestions:

- One student should design function `main`, which will call the other functions in the program. The remainder of the functions will be designed by other members of the team.
- The requirements of the program should be analyzed so each student is given about the same work load.
- The parameters and return types of each function should be decided in advance.
- The program can be implemented as a multi-file program, or all the functions can be cut and pasted into the main file.

Here is the assignment: Write a program that can be used by a small theater to sell tickets for performances. The theater's auditorium has 15 rows of seats, with 30 seats in each row. The program should display a screen that shows which seats are available and which are taken. For example, the following screen shows a chart depicting each seat in the theater. Seats that are taken are represented by an * symbol, and seats that are available are represented by a # symbol:

```

                Seats
            123456789012345678901234567890
Row 1      ***#####*#####*#####
Row 2      #####*#####*#####*#####
Row 3      **#####*#####*#####
Row 4      **#####*#####*#####
Row 5      *****#####*#####
Row 6      #####*#####*#####*#####
Row 7      #####*#####*#####*#####
Row 8      *****#####*#####
Row 9      #####*#####*#####*#####
Row 10     #####*#####*#####*#####
Row 11     *****#####*#####
Row 12     #####*#####*#####*#####
Row 13     *****#####*#####
Row 14     #####*#####*#####*#####
Row 15     #####*#####*#####*#####

```

Here is a list of tasks this program must perform:

- When the program begins, it should ask the user to enter the seat prices for each row. The prices can be stored in a separate array. (Alternatively, the prices may be read from a file.)
- Once the prices are entered, the program should display a seating chart similar to the one shown above. The user may enter the row and seat numbers for tickets being sold. Every time a ticket or group of tickets is purchased, the program should display the total ticket prices and update the seating chart.
- The program should keep a total of all ticket sales. The user should be given an option of viewing this amount.
- The program should also give the user an option to see a list of how many seats have been sold, how many seats are available in each row, and how many seats are available in the entire auditorium.

Input Validation: When tickets are being sold, do not accept row or seat numbers that do not exist. When someone requests a particular seat, the program should make sure that seat is available before it is sold.